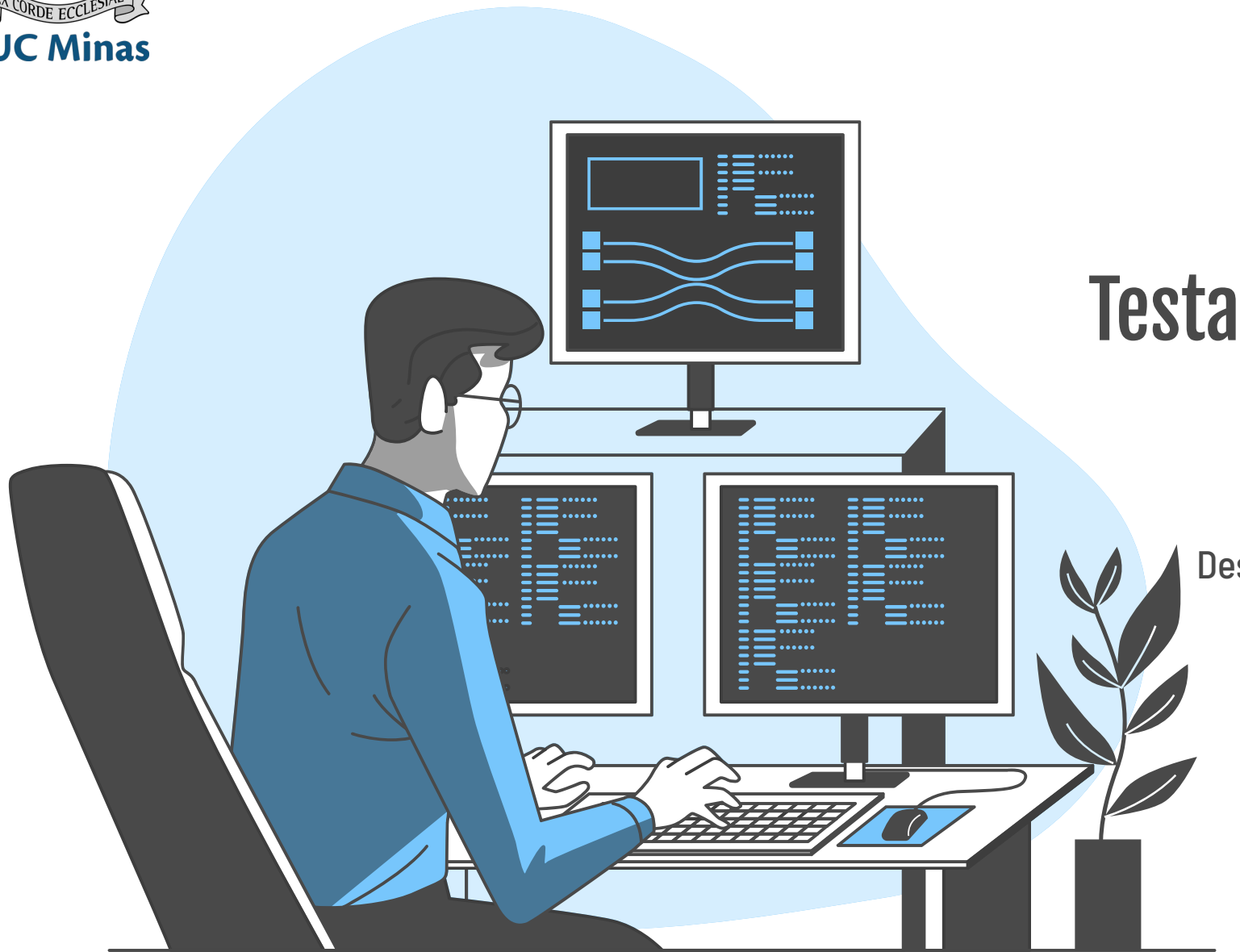
PUC Minas



Aula 09 · FullStack

Testando a integração: JUnit, MockMvc e Vitest

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Entender a **pirâmide de testes** em um projeto full stack
2. Testar services com **JUnit 5 + Mockito**
3. Testar controllers e o formato do JSON com **MockMvc**
4. Testar a configuração de **CORS**
5. Testar componentes e services do React com **Vitest** e **Testing Library**

Antes de começar

- Aulas 04 e 05 desta pasta
- Back-end, aula 04 (camadas)

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Por que testar a integração

02 Back-end: testando o service

03 Back-end: testando o controller com MockMvc

04 Back-end: testando o CORS

05 Front-end: Vitest e Testing Library

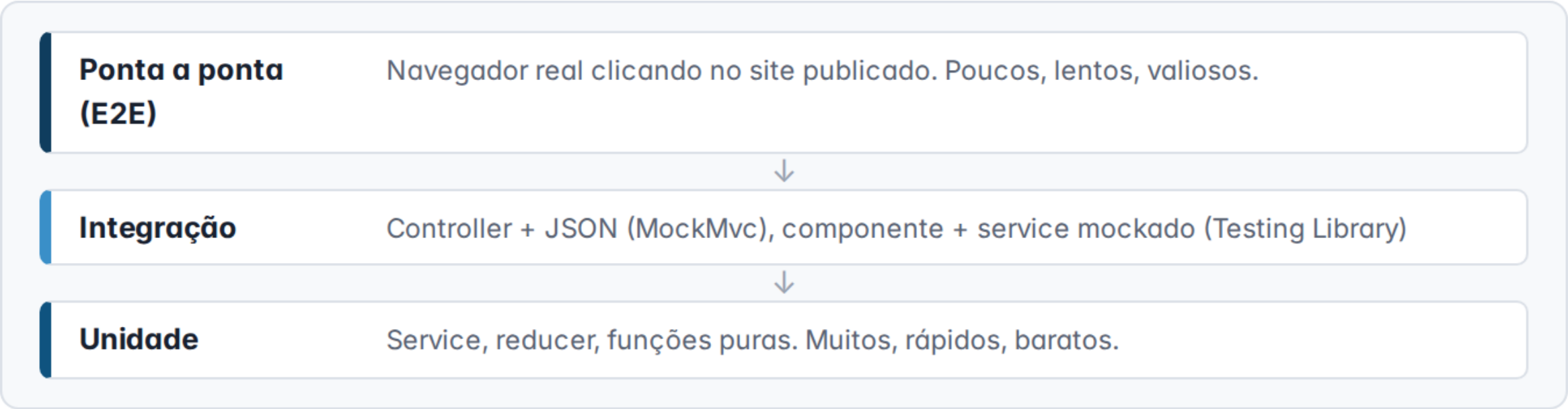
06 Aprofundando: testes no dia a dia

01

Por que testar a integração

Por que testar a integração

Em um projeto full stack, uma mudança pequena no back (renomear `nome` para `nomeCompleto` no JSON) quebra o front sem que ninguém perceba até alguém abrir a tela. Testes automatizados verificam o **contrato** entre as partes a cada alteração, em segundos.



Pirâmide de testes: muitos testes rápidos na base, poucos testes lentos no topo.

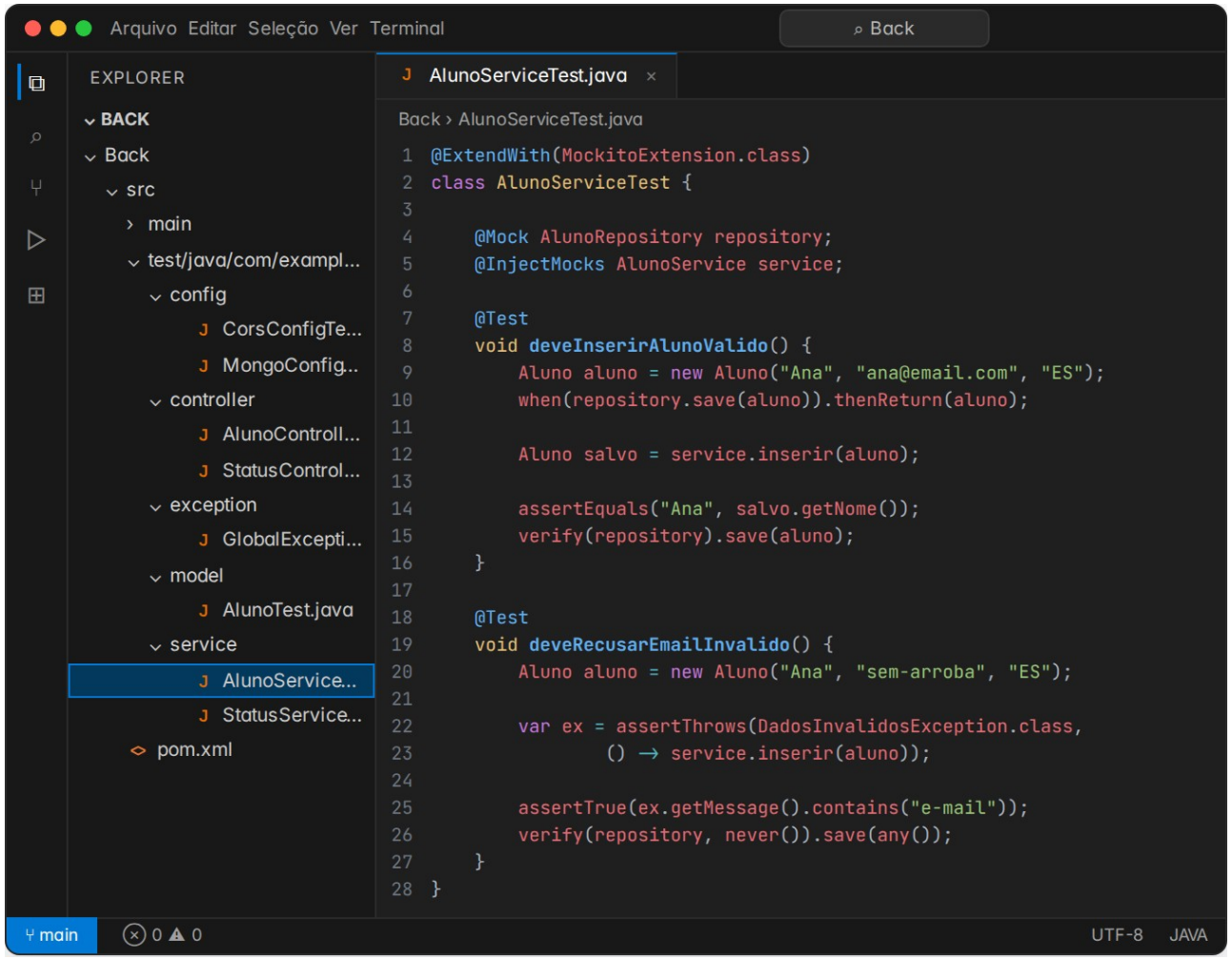
Por que testar a integração

Camada	Ferramenta	O que garante
Service (back)	JUnit 5 + Mockito	Regras de negócio e validações
Controller (back)	`@WebMvcTest` + MockMvc	Rotas, status HTTP e formato do JSON
Configuração (back)	MockMvc	CORS libera a origem certa
Service (front)	Vitest + `fetch` simulado	URLs, métodos e tratamento de erro
Componente (front)	Vitest + Testing Library	O que o usuário vê e faz

02

Back-end: testando o service

Back-end: testando o service



O projeto `Exemplo_Integracao/Back` já vem com testes em `src/test/java`, um para cada classe importante. O service é testado **sem banco**: o repository é substituído por um *mock* do Mockito.

Testes do `Exemplo_Integracao`: uma classe de teste para cada classe do código.

Back-end: testando o service

Anotação / método	Significado
`@ExtendWith(MockitoExtension.class)`	Liga o Mockito no JUnit 5
`@Mock`	Cria um objeto falso da interface
`@InjectMocks`	Cria o objeto real, injetando os mocks no construtor
`when(...).thenReturn(...)`	Define o que o mock devolve
`verify(mock).metodo(...)`	Confere que o método foi chamado
`assertThrows`	Confere que uma exceção foi lançada

03

Back-end: testando o controller com MockMvc

Back-end: testando o controller com MockMvc

O `@WebMvcTest` sobe **só a camada web** (controllers, conversão JSON, tratamento de erros), sem banco e sem Tomcat. O `MockMvc` simula requisições HTTP e permite conferir status e campos do JSON. É o teste que protege o **contrato** com o front.

controller/AlunoControllerTest.java

```
@WebMvcTest(AlunoController.class)
@Import(GlobalExceptionHandler.class)
class AlunoControllerTest {

    @Autowired MockMvc mvc;
    @MockitoBean AlunoService service;          // substitui o bean real

    @Test
    void listarDevolveJsonComOsCamposQueOFrontUsa() throws Exception {
        when(service.listar(null)).thenReturn(List.of(new Aluno("1", "Ana", "ana@email.com",
"ES")));

        mvc.perform(get("/api/alunos"))
           .andExpect(status().isOk())
           .andExpect(jsonPath("$[0].id").value("1"))
```

Back-end: testando o controller com MockMvc (cont.)

controller/AlunoControllerTest.java

```
            .andExpect(jsonPath("$[0].nome").value("Ana"))
            .andExpect(jsonPath("$[0].email").value("ana@email.com"));
    }

    @Test
    void erroDeValidacaoDevolve400ComMensagem() throws Exception {
        when(service.inserir(any())).thenThrow(new DadosInvalidosException("O nome do aluno é obrigatório."));

        mvc.perform(post("/api/alunos").contentType(MediaType.APPLICATION_JSON).content("{\"nome\":\"\"}"))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.mensagem").value("O nome do aluno é obrigatório."));
    }
}
```

Back-end: testando o controller com MockMvc

Observação

No Spring Boot 4 o `@MockBean` antigo foi substituído por **`@MockitoBean`** (pacote `org.springframework.test.context.bean.override.mockito`). E o `@WebMvcTest` vem do starter `spring-boot-starter-webmvc-test`, que já está no `pom.xml` do `Exemplo_Integracao`.

04

Back-end: testando o CORS

Back-end: testando o CORS

O erro de CORS só aparece no navegador, mas dá para testá-lo: basta simular a requisição com o cabeçalho `Origin` e conferir o `Access-Control-Allow-Origin` da resposta. É o que faz o `CorsConfigTest` dos projetos.

config/CorsConfigTest.java

```
@WebMvcTest(StatusController.class)
@Import({CorsConfig.class, CorsProperties.class})
@TestPropertySource(properties =
"app.cors.allowed-origins=http://localhost:5173,https://*.vercel.app")
class CorsConfigTest {

    @Autowired MockMvc mvc;
    @MockitoBean StatusService statusService;

    @Test
    void liberaOFrontLocal() throws Exception {
        mvc.perform(options("/api/status")
                .header("Origin", "http://localhost:5173")
```

Back-end: testando o CORS (cont.)

config/CorsConfigTest.java

```
                .header("Access-Control-Request-Method", "GET"))
            .andExpect(header().string("Access-Control-Allow-Origin", "http://localhost:5173"));
    }

    @Test
    void bloqueiaOrigemDesconhecida() throws Exception {
        mvc.perform(options("/api/status")
                .header("Origin", "https://site-malicioso.com")
                .header("Access-Control-Request-Method", "GET"))
            .andExpect(status().isForbidden());
    }
}
```

Back-end: testando o CORS

Terminal

```
aluno@pc:~/Exemplo_Integracao/Back$ ./mvnw test
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 -- in AlunoServiceTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0 -- in AlunoControllerTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- in CorsConfigTest
[INFO] Tests run: 27, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Rodando todos os testes do back. No VS Code, a aba **Testing** (ícone de frasco) mostra e roda cada teste individualmente.

05

Front-end: Vitest e Testing Library

Front-end: Vitest e Testing Library

O **Vitest** é o executor de testes do ecossistema Vite: mesma configuração, mesma velocidade. A **Testing Library** renderiza componentes e encontra elementos como o usuário encontraria: pelo texto, pelo papel (botão, campo) e pelo rótulo.

Instalação (dentro de Front/)

```
npm install -D vitest jsdom @testing-library/react @testing-library/user-event
@testing-library/jest-dom
```

Front-end: Vitest e Testing Library

vite.config.js

```
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";

export default defineConfig({
  plugins: [react()],
  test: {
    environment: "jsdom",       // navegador simulado
    setupFiles: "./src/setupTests.js",
    globals: true,
  },
});
```

setupTests.js e scripts

```
// src/setupTests.js
import "@testing-library/jest-dom/vitest";

// package.json
"scripts": {
  "test": "vitest",
  "test:run": "vitest run",
  "coverage": "vitest run --coverage"
}
```

Testando o service do front

Simulamos o `fetch` para conferir **qual URL e qual método** o service chama, sem precisar do Spring Boot rodando:

src/services/alunoService.test.js

```
import { describe, it, expect, vi, beforeEach } from "vitest";
import { criarAluno, listarAlunos } from "./alunoService";

describe("alunoService", () => {
  beforeEach(() => { global.fetch = vi.fn(); });

  it("lista alunos com filtro por nome", async () => {
    fetch.mockResolvedValue({ ok: true, status: 200, json: async () => [{ id: "1", nome: "Ana" }] });

    const alunos = await listarAlunos("ana");

    expect(fetch).toHaveBeenCalledWith("http://localhost:8080/api/alunos?nome=ana", expect.anything());
    expect(alunos[0].nome).toBe("Ana");
  });

  it("repassa a mensagem de erro do Spring", async () => {
    fetch.mockResolvedValue({ ok: false, status: 400, json: async () => ({ mensagem: "E-mail inválido." }) });

    await expect(criarAluno({ nome: "Ana", email: "x" })).rejects.toThrow("E-mail inválido.");
  });
});
```

Testando um componente

src/components/AlunoForm.test.jsx

```jsx
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import AlunoForm from "./AlunoForm";

it("envia os dados digitados ao clicar em Salvar", async () => {
  const onSalvar = vi.fn();
  render(<AlunoForm onSalvar={onSalvar} />);

  await userEvent.type(screen.getByPlaceholderText("Nome"), "Ana Souza");
  await userEvent.type(screen.getByPlaceholderText("E-mail"), "ana@email.com");
  await userEvent.click(screen.getByRole("button", { name: /salvar/i }));

  expect(onSalvar).toHaveBeenCalledWith(expect.objectContaining({ nome: "Ana Souza", email: "ana@email.com" }));
});

it("mostra erro quando o nome está vazio", async () => {
  render(<AlunoForm onSalvar={vi.fn()} />);
  await userEvent.click(screen.getByRole("button", { name: /salvar/i }));
  expect(screen.getByText("Informe o nome.")).toBeInTheDocument();
});
```

Testando um componente

Terminal

```
aluno@pc:~/Exemplo_Integracao/Front$ npm test

 ✓ src/services/alunoService.test.js (2 tests) 6ms
 ✓ src/components/AlunoForm.test.jsx (2 tests) 148ms

 Test Files  2 passed (2)
      Tests  4 passed (4)
   Duration  1.32s

 PASS  Waiting for file changes...
       press h to show help, press q to quit
```

O Vitest fica observando os arquivos: salvou, ele roda de novo os testes afetados.

Testando um componente

Consulta da Testing Library	Encontra	Prioridade
`getByRole("button", { name })`	Pelo papel acessível e nome	1ª opção
`getByLabelText("E-mail")`	Campo pelo ``	Formulários
`getByPlaceholderText`	Pelo placeholder	Quando não há label
`getByText`	Pelo texto visível	Mensagens
`getByTestId`	Por `data-testid`	Último recurso

Testando um componente

Dica

Se `getByRole("button", { name: /salvar/i })` não encontra o botão, provavelmente ele não tem texto acessível. Corrigir isso melhora o teste **e** a acessibilidade da tela.

06

Aprofundando: testes no dia a dia

Boas práticas

Um comportamento por teste

O nome do método diz o que se espera: `deveRecusarEmailInvalido`.

Arrange, Act, Assert

Prepare os dados, execute a ação, confira o resultado. Três blocos claros.

Teste o contrato

No controller, confira os **nomes dos campos** do JSON que o front usa.

Rápidos e independentes

Nenhum teste depende de outro nem do banco real.

Teste de unidade x teste de integração

Unidade

- Uma classe isolada, dependências mockadas
- Milissegundos
- Mostra exatamente onde está o erro

vs

Integração

- Várias peças juntas (Spring + JSON, componente + service)
- Segundos
- Pega erros de “encaixe” entre as partes

Aprofundando: testes no dia a dia

Teste com banco real em container (Testcontainers + Docker)

```
@SpringBootTest
@AutoConfigureMockMvc
@Testcontainers
class AlunoIntegracaoTest {

    @Container
    @ServiceConnection                          // o Spring aponta para este Mongo
    static MongoDBContainer mongo = new MongoDBContainer("mongo:7");

    @Autowired MockMvc mvc;

    @Test
    void criaEListaAlunoDeVerdade() throws Exception {
        mvc.perform(post("/api/alunos").contentType(MediaType.APPLICATION_JSON)
                .content("{\"nome\":\"Ana\",\"email\":\"ana@email.com\"}"))
           .andExpect(status().isCreated());
        mvc.perform(get("/api/alunos")).andExpect(jsonPath("$[0].nome").value("Ana"));
    }
}
```

Aprofundando: testes no dia a dia

Comando	O que faz
`./mvnw test`	Roda todos os testes do back
`./mvnw test -Dtest=AlunoServiceTest`	Roda uma classe só
`npm test`	Vitest em modo observação
`npm run test:run`	Roda uma vez e sai (para CI)
`npm run coverage`	Relatório de cobertura (instale `@vitest/coverage-v8`)

Aprofundando: testes no dia a dia

Na prática

Na pasta Deploy (aula 09), estes mesmos comandos rodam automaticamente no **GitHub Actions** a cada push. Se um teste falhar, o deploy não acontece.

Aprofundando: testes no dia a dia

1. Antes de mudar o JSON de uma resposta, procure no front quem usa aquele campo.
2. Atualize o teste do controller com o novo nome do campo (ele vai falhar).
3. Mude o código do back até o teste passar.
4. Atualize o service/componente do front e seus testes.
5. Rode `./mvnw test` e `npm run test:run` antes do push.

07

Quiz de revisão

Pergunta

Qual a base da pirâmide de testes?

A Testes ponta a ponta

B Testes de unidade, rápidos e numerosos

C Testes manuais

D Testes de carga

Resposta

Qual a base da pirâmide de testes?

A Testes ponta a ponta

✓ Testes de unidade, rápidos e numerosos

C Testes manuais

D Testes de carga

Por quê: E2E ficam no topo: poucos e lentos.

Quiz · questão 2 de 5

Pergunta

O que o `@WebMvcTest` sobe?

A Toda a aplicação e o banco

B Só a camada web (controllers, JSON, erros)

C Só o banco

D O Tomcat real

Resposta

O que o `@WebMvcTest` sobe?

A. Toda a aplicação e o banco

✓ Só a camada web (controllers, JSON, erros)

C. Só o banco

D. O Tomcat real

Por quê: O service é substituído por @MockitoBean.

Quiz · questão 3 de 5

Pergunta

Que teste protege o nome dos campos do JSON que o front usa?

A Teste do reducer

B MockMvc com jsonPath no controller

C Teste de CSS

D Teste de carga

Quiz · questão 3 de 5

Resposta

Que teste protege o nome dos campos do JSON que o front usa?

A Teste do reducer

✓ MockMvc com jsonPath no controller

C Teste de CSS

D Teste de carga

Por quê: Se alguém renomear o campo, o teste falha.

Pergunta

Como testar o service do front sem o Spring rodando?

A Impossível

B Simulando o fetch com vi.fn()

C Desligando o CORS

D Usando o Insomnia

Resposta

Como testar o service do front sem o Spring rodando?

A Impossível

✓ Simulando o fetch com vi.fn()

C Desligando o CORS

D Usando o Insomnia

Por quê: Confira URL, método e tratamento de erro.

Pergunta

Qual consulta da Testing Library é a primeira opção?

A getByTestId

B getByRole

C querySelector

D getElementById

Resposta

Qual consulta da Testing Library é a primeira opção?

A getByTestId

✓ getByRole

C querySelector

D getElementById

Por quê: Encontra elementos como o usuário e tecnologias assistivas.

Fechamento

Resumo da aula

- ✓ Pirâmide: muitos testes de unidade, alguns de integração, poucos de ponta a ponta.
- ✓ Service: JUnit 5 + Mockito, sem banco (`@Mock`, `@InjectMocks`, `verify`).
- ✓ Controller: `@WebMvcTest` + MockMvc + `jsonPath` protegem o contrato com o front; `@MockitoBean` no Boot 4.
- ✓ CORS testável: requisição `OPTIONS` com `Origin` e conferência do `Access-Control-Allow-Origin`.
- ✓ Front: Vitest + Testing Library; simule o `fetch` e encontre elementos como o usuário.

Exercícios

1. Escreva três testes para o seu service: caso válido, campo obrigatório vazio e id inexistente.

2. Escreva um teste MockMvc que falhe se o campo `nome` for renomeado no JSON.

3. Configure o Vitest no seu front e teste o `httpClient` com sucesso, erro 400 e falha de rede.

4. Teste o formulário: digitar, clicar em salvar e conferir a função chamada.

Para ir além

- Pastas `src/test` de `FullStack/Exemplo_Integracao/Back` e `FullStack/AraCoffee/Back`
- Projeto `SpringBoot/Tests_JUnit_Mockito`
- vitest.dev e testing-library.com/docs/react-testing-library/intro

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 09?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web